



## HACKFEST 2026

### Problem Statement Documentation & Technical Submission

*(Template / Dummy Format – for Team Reference)*

|                               |                                                          |
|-------------------------------|----------------------------------------------------------|
| <b>Team ID</b>                | KH131                                                    |
| <b>Team Name</b>              | KALA DHUA                                                |
| <b>Team Leader Name</b>       | Ayush Pathak                                             |
| <b>Problem Statement No.</b>  | PS20                                                     |
| <b>Problem Statement Name</b> | Agentic Disaster Relief & Emergency Resource Coordinator |
| <b>Date of Submission</b>     | [11 / 09 / 2026]                                         |



## SECTION 1 — TEAM DETAILS

**Team ID**

KH131

**Team Name**

Kala Dhua

**Team Leader Name**

Ayush Pathak

**Team Members Name(s)**

*List the names of all other team members.*

Shreeviraj Matala

**Problem Statement Number**

PS20

**Problem Statement Name**

Agentic Disaster Relief & Emergency Resource Coordinator

## SECTION 2 — UNDERSTANDING OF THE PROBLEM

### Problem Statement

*Explain the problem your team is solving, in your own words.*

When a city floods, the thing that runs out first is not equipment. Pune has boats, pumps, ambulances, buses and rescue teams sitting in depots across the city. What breaks is coordination. Reports arrive over phone calls, WhatsApp groups, ward offices and news channels, and four or five of them are usually describing the same flooded junction. No single desk can say which reports are the same event, which are exaggerated, and which is the one that actually needs a boat in the next ten minutes.

The situation also refuses to hold still. A road goes under water, so the ambulance that was eight minutes away is now twenty five. A distress call comes in from a ward that was quiet an hour ago. The plan that was correct at 10:00 is wrong by 10:15, and rebuilding it over radio takes longer than the disaster gives you.

What follows is the exact failure PS20 names: duplicated effort and misdirected effort. Two agencies send teams to the same bridge while a ward with 2,300 people waits for the one that never got sent. Afterwards nobody can reconstruct who decided what, or on whose authority.

We are building the coordination layer that sits between incoming information and the people who act on it. It takes messy, contradictory and incomplete reports, works out which ones describe the same incident and how much each can be trusted, allocates limited resources against that picture using a solver instead of intuition, reallocates the moment the picture changes, and keeps a permanent record of every decision and the clause that authorised it.

### Problem Analysis / Background

*What is the current situation or existing problem? Who is affected by it?*

Today a municipal control room during a flood runs on a phone bank, a whiteboard, a WhatsApp group and a spreadsheet. Incoming reports are logged as separate rows even when five of them are the same waterlogged street. Resource status is whatever the last radio call said. Allocation is done by whoever is most senior in the room, from memory, under time pressure. Nothing is timestamped in a way that survives the night.

That has four concrete consequences. Duplicate reports inflate how bad a zone looks, so attention drifts there. Two agencies dispatch to the same incident because neither can see the other's assignments. Resources get committed to reports that later turn out to be wrong or malicious, and they are not available when the real call comes. And the after action review has no reliable timeline to work from, so the same mistakes repeat next monsoon.

The people affected are, first, residents of low lying wards who wait for help that was sent somewhere else. Then ward officers, who are asked to make high stakes calls on partial information and are personally accountable for them. Then field teams from fire services, police, NDRF, SDRF, municipal works and NGOs, who are dispatched without knowing who else is already on scene. And finally the Municipal Commissioner, who holds the delegation for the largest actions such as mass evacuation and has to sign them off with whatever evidence reaches the desk.

There is one more thing that is usually ignored. Disaster information is not just incomplete, it is sometimes hostile. Rumours spread faster than verified reports, and a flood of identical fake reports from newly created accounts is enough to pull scarce rescue units away from where they are needed. Any system that accepts public reports and then moves real vehicles has to treat that as a threat model, not an edge case.

### Proposed Solution

*Explain your proposed solution and how it addresses the selected Problem Statement.*

We are building Indradhanu OS, an agentic disaster coordination platform for a city. It runs across the whole lifecycle: predict and pre position before the event, verify and coordinate during it, and replay and learn after it. The prediction side already exists in our codebase as hazard adapters for flood, heat, fire, air quality and seismic risk. PS20 sits on top of that as the response engine.

The design rule we hold to throughout is that the language model reasons and explains, but never produces a number. Risk scores come from the hazard adapters. Allocation comes from a CP-SAT constraint solver. Severity, priority and trust come

---

from deterministic scoring functions. Authority comes from a policy corpus of delegation clauses. If the model is unavailable the system still runs and reports that it degraded, which is also our honest answer to the question of what happens when an LLM is wrong.

Four agents, not fifteen. A Triage agent decides what is happening and whether it can be believed. A Needs agent works out what a zone actually requires. A Coordination agent picks the responding agency and proposes a strategy. A Communication agent writes the citizen alert, the field instruction and the agency request in English, Hindi and Marathi. Everything numeric underneath them is deterministic and inspectable.

Two pieces make it genuinely dynamic rather than a one shot planner. First, every state change in the system is written to an append only event log, which serves at once as the PS20 activity and audit log, the trigger for replanning, and the tape for replay. Second, allocation is incremental: the solver is given what is already committed and a switching cost, so a new urgent report can pull a unit off a lower priority job, but a boat that is most of the way to a rescue does not get turned around for no reason. The console shows the difference between the old plan and the new one, with the reason for each change.

On top of that sits the part PS20 asks for by name and that most systems skip: duplicate effort detection. Incoming reports are clustered into incidents using geography, time, category and text similarity, so five reports about one bridge become one incident with a higher confidence rather than five competing calls for help. Separately, the system flags when two agencies have dispatched into the same incident cluster, which is the form of duplication that actually wastes rescue capacity.

---

## SECTION 3 — RESEARCH

### Research Conducted

*Research, references, datasets, papers, websites, APIs, documentation, or other resources used.*

Hazard and weather data. We use the Open-Meteo forecast, flood and air quality APIs, which are free for non commercial use and need no key, with NASA FIRMS for active fire detections and data.gov.in for Indian open datasets. These are already wired into the ingestion layer with caching and a fallback path.

Optimisation and routing. Google OR-Tools CP-SAT documentation for the assignment formulation, and OSRM for real road travel times and distances rather than straight line estimates. We read the OR-Tools assignment and vehicle routing guides to decide how to express coverage as an objective term instead of a hard constraint, which is what lets the solver tell us honestly what it could not cover.

Relief standards and alerting formats. The Sphere Handbook minimum standards were used as the basis for the needs assessment ratios, such as water per person per day and shelter area per person, so that resource baskets are grounded in something published rather than invented. The Common Alerting Protocol (CAP) specification informed the structure of our alert objects, including severity, urgency, certainty and the area they apply to.

Data, policy and platform. CrisisNLP and CrisisMMD, which are public labelled collections of disaster related social posts, for testing report classification and deduplication behaviour. NDMA guidelines and a municipal disaster management plan for the delegation matrix that drives our policy gate. PostGIS documentation for geography types and distance queries, pgvector for text similarity, and Supabase and FastAPI documentation for the platform itself.

### Existing Solutions / Similar Work

*Mention existing solutions/projects related to your idea and how yours is different or improved.*

Ushahidi and Sahana Eden are the closest established projects. Ushahidi is strong at crowdsourced incident mapping and Sahana Eden at relief inventory and organisation registries. Both are essentially record keeping and visualisation systems: a human still reads the map and decides who goes where. Neither performs constrained resource optimisation, and neither reallocates automatically when the situation changes.

On the government side, NDMA SACHET and IMD warnings are one way broadcast. They tell citizens a hazard is coming and do an important job, but they stop at the alert. They do not model resource availability, they do not assign units to incidents, and they have no concept of an incident that four different people reported.

Google Public Alerts and similar crisis response products aggregate and distribute official warnings. Again the loop ends at information delivery. Commercial incident command tools such as those used by large fire services do handle dispatch, but they assume a single agency with clean, authenticated inputs from its own staff, not a mixed stream of citizen reports of unknown reliability across several organisations.

What we do differently is close the loop and defend it. Report to verified incident to assessed need to solver backed allocation to dispatch to automatic reallocation, with three things bolted in that we have not found combined elsewhere: a trust score that decides whether a report is allowed to move a vehicle at all, duplicate effort detection across agencies, and a policy gate where an action issues because a delegation clause permits it rather than because a model was confident. The whole run is replayable afterwards, so we can measure our own decisions against a manual baseline instead of asserting that we are better.

### References / Sources

*Provide links to the important sources used during research.*

Open-Meteo APIs: <https://open-meteo.com/en/docs> | Flood API: <https://open-meteo.com/en/docs/flood-api> | Air Quality API: <https://open-meteo.com/en/docs/air-quality-api>

Google OR-Tools CP-SAT: <https://developers.google.com/optimization/cp> | OSRM routing engine: <http://project-osrm.org/> | PostGIS: <https://postgis.net/documentation/> | pgvector: <https://github.com/pgvector/pgvector>



---

Sphere Handbook: <https://spherestandards.org/handbook/> | Common Alerting Protocol: <https://docs.oasis-open.org/emergency/cap/v1.2/CAP-v1.2.html> | NDMA guidelines: <https://ndma.gov.in/> | NASA FIRMS: <https://firms.modaps.eosdis.nasa.gov/> | data.gov.in: <https://data.gov.in/> | CrisisNLP: <https://crisisnlp.qcri.org/>

---

## SECTION 4 — TECHNICAL DOCUMENTATION

### Technology Stack

*Languages, frameworks, libraries, databases, APIs, cloud services, AI/ML models, etc.*

Backend: Python 3.12, FastAPI, Pydantic v2 for typed settings and wire models, asyncpg with a pooled connection, structlog for JSON logging, Uvicorn, Docker. Optimisation with Google OR-Tools CP-SAT and a greedy nearest first fallback. Routing with OSRM.

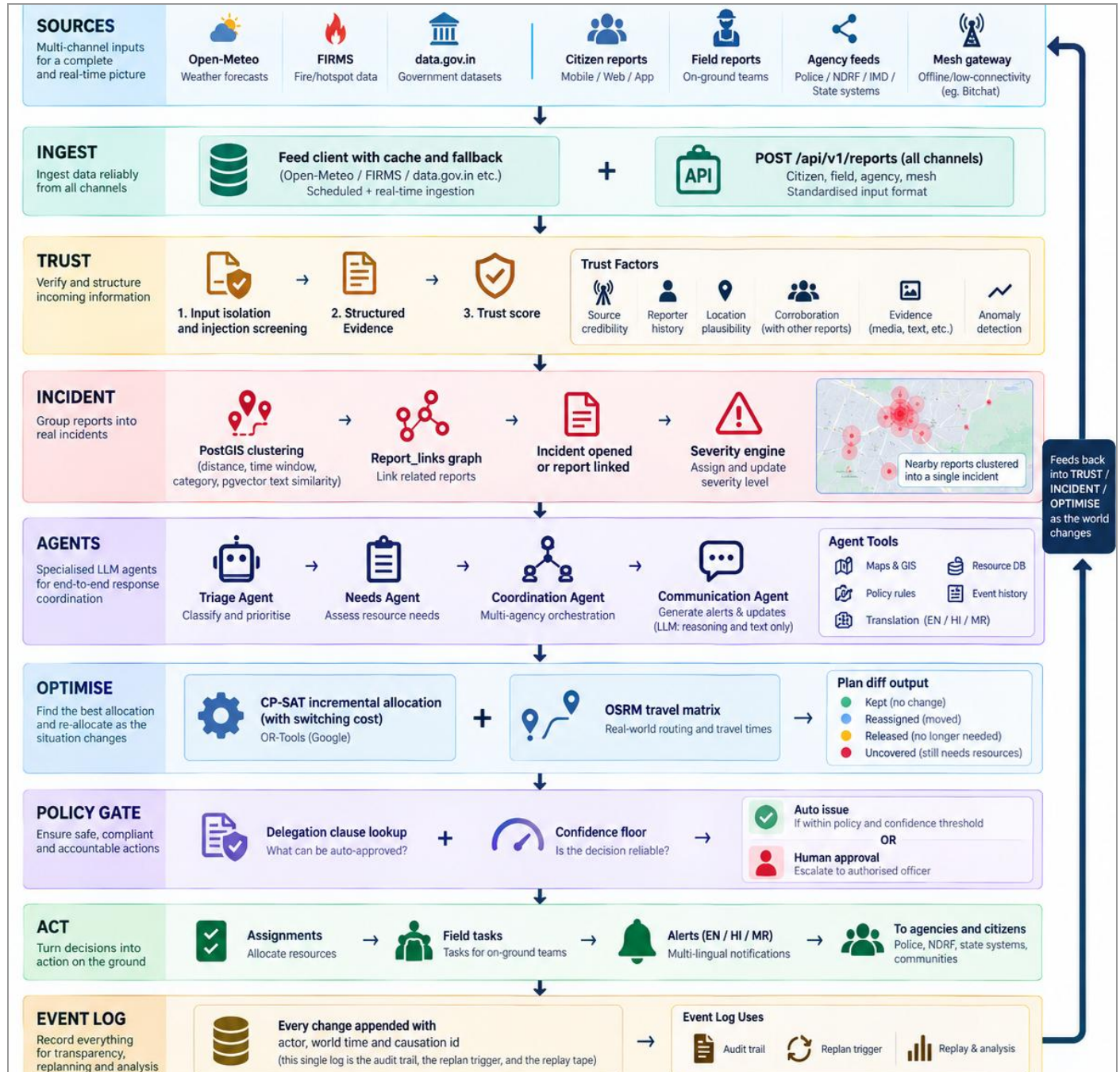
Data: Supabase Postgres 17 with PostGIS for ward boundaries, incident and resource geography and real distance queries, and pgvector for text and policy clause similarity. Row level security, Supabase Auth and JWT verification for the three roles (citizen, field operator, ward officer or commissioner). Schema covers wards, lifelines, resources, hazard runs, ward risks, citizen reports, incidents, allocation plans, assignments, decisions, alerts, field tasks, agent runs and agent steps, plus the new append only event log.

Frontend: React 19, TypeScript, Vite, Tailwind CSS v4, shadcn/ui on Radix primitives, TanStack Query, Zustand, React Router, Mapbox GL for the map, Recharts for charts, Framer Motion for transitions. Three interfaces in one app: an operations console, a citizen portal and a field operator view. AI layer: Gemini 2.0 Flash in development and AWS Bedrock in production, behind a provider interface with a deterministic rule based fallback.

---

## Technical Flow / System Architecture Diagram

Attach / paste your architecture diagram below.



## Working / Implementation Approach

Explain how your system works from input to output.

**Input.** A report enters through one endpoint no matter where it came from: a resident on the citizen portal, a field operator on a phone, an agency feed, or a batch of queued messages from a device that was offline. It carries a ward, a category, a location, a note and optionally a photo. Before any agent sees the free text, the text is put through structured extraction and an injection screen, so that instructions hidden inside a report are treated as data and never as commands.

**Verification.** The report gets a trust score from six deterministic components: how credible the channel is, how that reporter has performed historically, whether the location is plausible given current hazard risk and elevation, how many independent

reports corroborate it, whether there is photo evidence, and whether it looks anomalous (bursts from one source, near identical text, impossible movement between reports). The score decides routing: high trust goes straight into the pipeline, medium trust asks for corroboration, low trust is queued and is not allowed to move a vehicle. High impact with low confidence always goes to a human.

Incident formation. The report is compared against open incidents nearby using a PostGIS distance query bounded by a category specific radius and time window, then scored on spatial proximity, time, category match and text similarity. Above a threshold it is linked automatically and the incident's report count and confidence rise. In the uncertain middle band the Triage agent adjudicates a single pair and its answer is stored with the link. Below the threshold, a new incident is opened. This is where five reports about one bridge collapse into one incident instead of five.

Assessment and allocation. The Needs agent turns the incident and its exposed population into a resource basket (boats, pumps, ambulances, medical teams, shelter beds, food kits), using published ratios rather than invented ones. A deterministic severity and priority engine scores it from population exposed, severity, vulnerability, time to impact, resource scarcity and confidence. OSRM builds the travel time matrix over the current road graph, with closed roads removed. CP-SAT then solves the assignment, minimising population weighted time to arrival, subject to one job per unit, kind compatibility and a forty five minute reachability limit, with coverage rewarded in the objective. It is given the current commitments and a switching cost, so it returns a plan difference rather than a fresh plan.

Authority and output. Every proposed action is matched to a delegation clause from the policy corpus. An action issues automatically only if the clause delegates it at that severity and confidence is above the floor; otherwise it waits for the officer who actually holds that delegation, and the interface names the clause that blocked it. Approved actions become assignments, field tasks and alerts. Every one of these steps appends to the event log with the event that caused it, so the console can show not only what the system did but why, and the whole run can be replayed afterwards to compare against a manual baseline.

### AI/ML Details (if applicable)

*Dataset, model, training approach, APIs/models used, evaluation, etc.*

Models and where they are allowed to act. Gemini 3.5 Flash in development and AWS Bedrock in production, behind one provider interface. The model is used for four things only: adjudicating ambiguous report links, mapping an incident to a resource basket with an explanation, proposing a coordination strategy and rationale, and drafting citizen, field and agency messages in English, Hindi and Marathi. It never emits a risk score, a severity, a priority, a trust score, an ETA or an allocation. Those come from hazard adapters, scoring functions, OSRM and CP-SAT.

Non LLM components. Hazard scoring adapters per hazard type, each with its own signal fetch, score and action policy. A deterministic severity and priority model. A six component trust score. Report deduplication using PostGIS geography distance, time decay, a category compatibility matrix and pgvector cosine similarity over report text. Policy clause retrieval is exact match on an action key by default, with a pgvector similarity path available as the clause corpus grows, because fuzzy similarity should not be what decides whether an evacuation was authorised.

Data and evaluation. No model training is planned. Public data is used for grounding and testing: Open-Meteo forecast and flood series, NASA FIRMS, data.gov.in datasets, and CrisisNLP or CrisisMMD for exercising classification and deduplication. Evaluation is by counterfactual replay. The same recorded stream of world events is pushed through three policies (nearest available first come first served, one shot optimisation at time zero, and our reactive system) and we compare median and p90 time from report to assignment, time to resolution, unmet demand weighted by population, duplicate deployments avoided, resource minutes saved by not dispatching to unverified reports, total travel distance, and reassignment churn. Load behaviour is measured separately at 100, 1,000, 10,000 and 100,000 synthetic reports.

### GitHub Repository Link

*Provide the team's public GitHub repository URL.*

<https://github.com/officialayush23/DisruptionOps>

## SECTION 5 — ORIGINALITY & PLAGIARISM DECLARATION

- ✓ I confirm that the solution submitted by our team is developed during the hackathon. We have not submitted a pre-built project as our primary solution. Any open-source code, APIs, datasets, libraries, templates, or AI-assisted development used by us have been appropriately acknowledged.

### Open-Source / External Resources Used

*GitHub repositories, code snippets, templates, APIs, datasets, models, or other external resources used.*

Backend libraries: FastAPI, Pydantic and pydantic-settings, asyncpg, structlog, orjson, httpx, Uvicorn, Google OR-Tools (CP-SAT). Database: Supabase Postgres with the PostGIS and pgvector extensions.

Frontend libraries: React, Vite, TypeScript, Tailwind CSS, shadcn/ui component templates built on Radix UI, TanStack Query, Zustand, React Router, Mapbox GL JS, Recharts, Framer Motion, Lucide icons, Sonner.

External APIs and data: Open-Meteo forecast, flood and air quality APIs; the public OSRM demo routing server; NASA FIRMS active fire data; data.gov.in open datasets; Google Gemini API and AWS Bedrock for language model inference. Standards referenced: Sphere Handbook minimum standards and the Common Alerting Protocol. All third party code is used under its own licence and is acknowledged here.

---

### AI Tools Used

*Mention AI tools used during development, if any, and briefly explain how they were used.*

Claude (Anthropic) and ChatGPT were used during development as coding assistants: scaffolding boilerplate such as repository query functions and React components, reviewing our CP-SAT model formulation, drafting SQL migrations, and reviewing code for mistakes. Every suggestion was read, edited and tested by us before it went into the repository.

GitHub Copilot was used for inline completion while writing code.

Gemini 3.5 Flash and AWS Bedrock are not development tools but runtime components of the product itself, used for the reasoning and text generation described in the AI/ML section. The architecture, the hazard scoring, the trust model, the deduplication logic, the solver formulation and the policy gate design are our own work.

---

## SECTION 6 — UPLOADS / ATTACHMENTS CHECKLIST

- ✓ Technical Flow / Architecture Diagram
- ✓ Research / Reference Document (if applicable)
- ✓ Project Documentation / Technical Report
- ✓ GitHub Repository Link
- ✓ Screenshots of Working Prototype
- ✓ Dataset / Sample Data (if applicable)
- ✓ Additional Diagrams (if any)

---

Team Leader Signature: \_\_\_\_\_ Ayush Pathak \_\_\_\_\_ Date: \_11/09/2026\_\_\_\_\_